

KUTO BARAN



HANA TECH CON 2026

THE ROLE OF INDEXES IN SAP

HANA PERFORMANCE



CONTENT

- Introduction to SAP HANA Indexes
- Determination and usage of SAP HANA Indexes
- Experiences (Examples) with SAP HANA Indexes: Performance
- Takeaways
- References



INTRODUCTION TO SAP HANA INDEXES

- **The Popular Misconception:**
- *"Since SAP HANA is an in-memory, column-oriented database, indexes are completely obsolete."* : SAP Note [2100010 - SAP HANA: Popular Misconceptions](#)

Source: SAP

INTRODUCTION TO SAP HANA INDEXES

Two SAP systems with identical data content (2019)

ORACLE

in the SAP database schema

- # tables : 118.618
- # indexes : 147.323

Without implicit indexes:

secondary indexes per table

On Oracle: about 0,24

On SAP HANA: 0,006

With HANA implicit indexes:

Oracle: 1,24 indexes per table

On SAP HANA: 4,08 indexes per table

S/4HANA (without custom optimizations and manual tuning regarding indexes)

in the SAP database schema

- # tables : 141.288
- # indexes : 142.131
- # implicit indexes: 433.871
 - Evaluation with a part of the SQL
„HANA_Indexes_Columns“:
(SAP note: [1969700 - SQL
Statement Collection for
SAP HANA](#)):
- # all indexes: 576.002

INTRODUCTION TO SAP HANA INDEXES

- **purpose of indexes in SAP HANA environments:**
 - Support of a primary key or unique key constraints
 - Efficient support of foreign key constraints
 - Optimization of central, performance-critical queries
 - Optimization of queries consuming a lot of CPU resources due to highly parallelized operations

INTRODUCTION TO SAP HANA INDEXES



- Tables in row store need indexes similarly like tables in classic relational databases
- Column store indexes:
 - Single-column Indexes
 - Light-weight implementations as inverted indexes built on top of the column structure
 - Multi-column Indexes
 - Stored as internal columns and inverted index, known as concat attributes
 - Can be useful, whenever only a combination of multiple columns offers sufficient selectivity, whereas individual columns do not.
 - Single-column indexes preferred over multi-column indexes in column store

INTRODUCTION TO SAP HANA INDEXES



Special analysis tools required for visibility:

- SQL: *"HANA_Tables_LargestTables"*
SQL: *"HANA_Indexes_Overview"*
- row store: tables -> shared memory, indexes -> heap memory

TABLE_NAME	DISK_GB	MEM_GB	TAB_MEM_GB	INDEXES	IND_MEM_GB
CDPOS	1045.29	934.08	399.41	1/0/9	534.66
SWWLOGHIST	305.25	321.21	106.04	3/0/9	215.17
ICLPARTOCC	263.21	269.04	70.67	1/0/8	198.36

...

TABLE_NAME	INDEX_NAME	C	INDEX_TYPE	MEM_TOT_GB	MEM_CONC_GB	SEL_COL_NAME	INDIV_COSTS
CDPOS	CDPOS~0	P	INVERTED VALUE	534.66	418.83	CHANGENR	3
ICLPARTOCC	ICLPARTOCC~0	P	INVERTED VALUE	198.34	160.82	CLAIM	30
SWWLOGHIST	SWWLOGHIST~0	P	INVERTED VALUE	197.58	166.65	WI_ID	239

...

INTRODUCTION TO SAP HANA INDEXES



- Implicit single column indexes when creating primary key
- Implicit single column indexes on compressed columns (compression type (e.g. CLUSTERED, INDIRECT, RLE ...) supporting a BLOCK index and the size of the single column index isn't larger than 10 % of the column size)

```
*****
* INDEX INFORMATION *
*****
```

SCHEMA_NAME	TABLE_NAME	INDEX_NAME	COLUMN_NAME	INDEX_TYPE	CONSTRAINT_NAME
SCHEMAT	ICLCLAIM	ICLCLAIM-0	CLIENT	INVERTED VALUE UNIQUE	PRIMARY KEY
SCHEMAT	ICLCLAIM	ICLCLAIM-H92	EXCLAIMNO	INVERTED VALUE	

```
*****
* COLUMN INFORMATION *
*****
```

TABLE_NAME	COLUMN_NAME	NUM_DISTINCT	SIZE_MB	LENGTH	DATA_TYPE	COMPRESSION	INDEX_TYPE
ICLCLAIM	ACTIVE	1	0.02	1	NVARCHAR	RLE	BLOCK
	CLAIMTYPE	64	55.93	4	NVARCHAR	INDIRECT	BLOCK
	CLEARDATE	1	0.02	8	NVARCHAR	PREFIXED	NONE
	CLIENT	1	0.02	3	NVARCHAR	RLE	BLOCK
	CLTYPEG	60	77.69	5	NVARCHAR	INDIRECT	BLOCK
	CLTYPEV	58	57.13	5	NVARCHAR	INDIRECT	BLOCK
	COMPLEX	1	0.01	1	NVARCHAR	PREFIXED	NONE
	COVERED	2	0.02	1	NVARCHAR	RLE	NONE
	GENERAT	9	38.87	5	NVARCHAR	INDIRECT	BLOCK

INTRODUCTION TO SAP HANA INDEXES

- Row Store:
 - BTREE [UNIQUE]
 - CPBTREE [UNIQUE] *SAP Note [2112604](#)*, BTREE with compressed prefix)
- Column Store:
 - FULLTEXT (*SAP Note [2800008](#)*)
 - FULLTEXT TEXT ANALYSIS
 - FUZZY SEARCH
 - HNSW VECTOR (*SAP Note [3463689](#)*)
 - INVERTED HASH [UNIQUE] (*SAP Note [2109355](#)*)
 - INVERTED INDIVIDUAL [UNIQUE] (*SAP Note [2600076](#)*): Default for HANA Cloud
 - INVERTED VALUE [UNIQUE]: Default for HANA on Premise

- different implementations of column store indexes:
 - FULL Indexes: Direct mapping from value IDs to row positions
 - BLOCK Indexes: mapping value IDs to specific blocks
- Distribution of indexes by index type in an S/4HANA system

	COUNT	INDEX_TYPE

	716	CPBTREE UNIQUE
	180	CPBTREE
	9	BTREE UNIQUE
	178.959	INVERTED VALUE
	226	FULLTEXT
	1	FULLTEXT TEXT ANALYSIS
	205	INVERTED INDIVIDUAL
	84	FUZZY SEARCH

- # explicit indexes : 180.380
- # Implicit indexes: 421.941

Source: SAP

DETERMINATION AND USAGE OF SAP HANA INDEXES

- **tools which can help to find useful indexes**
- HANA Cloud: Index Advisor
- HANA On Premise: No Index Advisor
- Root cause analysis for the affected statement
 - Explain plan and PlanViz
 - Collection of details for a specific SQL statement: „*HANA_SQL_StatementHash_DataCollector*“
 - HANA thread methods: ([2000002](#) and [2114710](#))
 - IndirectScanBvOutJob*
 - sse_icc_lib::mgetSearchi_AVX2impl
 - ...
 - SQL: „*HANA_Threads_ThreadSamples_FilterAndAggregation*“
 - Call stacks contain information like ([2313619 - How-To: Generating and Evaluating SAP HANA Call Stacks](#))
 - hex::cs::TableScanControlOp::run
 - sse_icc_lib::mgetSearchi_SSE4impl
 - ...
 - SQL: „*HANA_Threads_Callstacks_[History]*“

DETERMINATION AND USAGE OF SAP HANA INDEXES

General search for full column scans:

- Determination of Statements with Full Column Scans: SQL:
"HANA_Threads_ThreadSamples_FilterAndAggregation": (ONLY_FULL_SCAN_METHODS = 'X', 'HASH' AGGREGATE_BY, 'COUNT' ORDER_BY)
- HANA Configuration Minichecks:
 - SQL: "HANA_Configuration_MiniChecks"
 - Check ID M1125 ("Columns with many scanned records") $\geq$ 10 million scanned records per second and only top ten columns. You can also use other values as a threshold.
 - Check ID M1126 ("Row store tables with many scanned records") $\geq$ 20000 scanned records

M1125 Columns with many scanned records	DD03L.FIELDNAME (134 million/s)
M1125 Columns with many scanned records	FQM_FLOW.AMOUNT (65 million/s)
M1125 Columns with many scanned records	FQM_FLOW.BASE_AMOUNT (63 million/s)

DETERMINATION AND USAGE OF SAP HANA INDEXES

- Column Statistics for high number of scanned records:
 - SQL: "HANA_Tables_ColumnStore_Columns_Statistics", MIN_SCANNED_RECORDS_PER_S = 10000000, MIN_DIST_COUNT = 1000);
 - Be aware that a SAP HANA restart resets this counter to 0.**

TABLE_NAME	COLUMN_NAME	SCANNED_RECORDS	SCR_PER_S	INDEX_LOOKUPS	IL_PER...	MEM_SIZE_MB	RECORDS	DIST_COUNT	COMPRESSION	INDEX_TYPE
DD03L	FIELDNAME	1157610184536568	134284302	1155715	0.13	78.29	18037024	1181769	INDIRECT	FULL

TABLE_NAME	COLUMN_NAME	SCANNED_RECORDS	SCR_PER_S	INDEX_LOOKUPS	IL_PER...	MEM_SIZE_MB	RECORDS	DIST_COUNT	COMPRESSION	INDEX_TYPE
FQM_FLOW	AMOUNT	562139706414373	65198341	0	0.00	905.48	442128824	5053432	INDIRECT	NONE
FQM_FLOW	BASE_AMOUNT	546013869974388	63328027	0	0.00	786.91	442128824	3495548	INDIRECT	NONE

- General Column Statistics are also available:
 - SQL: "HANA_Tables_ColumnStore_Columns_Statistics"
 - SQL: „HANA_SQL_StatementHash_DataCollector"
 - SQL: „HANA_Tables_DataCollector"
- One typical reason for increased scan activities are missing indexes, but also other scenarios (e.g. a very unselective database request) are possible.

DETERMINATION AND USAGE OF SAP HANA INDEXES

- Remove of unused large non-unique concat attributes
 - MiniCheckID : M0455 → Unused large non-unique concat attributes
 - SQL: "HANA_Tables_ColumnStore_Columns_Statistics" (ONLY_UNUSED_CONCAT_ATTRIBUTES = 'X')
- Indexed Columns with Least Index Lookups: SQL: „HANA_Tables_ColumnStore_Columns_Statistics,,

TABLE_NAME	COLUMN_NAME	SCR_PER_S	INDEX_LOOKUPS	DIST_COUNT
SBCMCONT1	SRTF2	2088780	0	21250
OBJH	OBJECTNAME	10778	0	87959
DBTABLOG	TABNAME	241104	0	71955
ICLCLAIM	ACHANGETIME	3841674	2	54350425
FKKVKP	GPARG	12259	5	107017
BALHDR	ALDATE	42853	54	6274
FKKMAKO	AUSBL	250463	86	48362415
ICLACTIVITY	ACTIVITY	29282	208	5390
BUTADRSEARCH	ADDRNUMBER	11101	474	20140868
ICLPROCURE	\$PDOCCAT\$STATUS\$DELETED\$DUEDATES\$	889906	1859	83937
SWWLOGHIST	METH_EDATE	298754	2561	7184
POC_D_PRC_STP_IN	PROCESS_ID	10537	3092	16925469

DETERMINATION AND USAGE OF SAP HANA INDEXES

- **Where can I see if an index is used by a certain SQL statement?**
- PlanViz
- Explain Plan: The Explain Plan can be collected in several different ways:
 - By running the EXPLAIN PLAN SQL statement
 - EXPLAIN PLAN [**SET** STATEMENT_NAME = '<statement_name>'] **FOR** <Query>
 - EXPLAIN PLAN [**SET** STATEMENT_NAME = '<statement_name>'] **FOR SQL PLAN** CACHE ENTRY <sql_plan_id>
 - SELECT * from explain_plan_table where STATEMENT_NAME = '<statement_name>']
 - In the SQL console of SAP HANA studio oder HANA Cockpit (Analyze--> Explain Plan)
 - From within DBACOCKPIT
 - With ABAP ST12 Traces

DETERMINATION AND USAGE OF SAP HANA INDEXES

ABAP DBACOCKPIT

ABAP ST12 or ST05 Traces

SQL Editor

Full Screen On/Off Execute Execution Trace Explain Replace ' ST05 Structure Identical Statements

History: Query Name:

SAP HANA database: Database Ad...

- Current Status
- Performance
- Configuration
- Jobs
- Diagnostics
 - Audit Log
 - Missing Tables and Indexes
 - EXPLAIN**
 - Self-Monitoring
 - Consistency Checks Errors History
 - SQL Editor**
 - Execution Trace
 - Tables/Views

Input Query Result

```
select * from BALDAT where LOG_HANDLE=?
```

For Recorded Statement
For Statement to Be Entered

Executions	Redundancy	Id	pry	Records	Dur./Exec.	Rec./Exec.	Dur./Rec.	Dur./Rec.	Dur./Rec.	Lngh	Buffer	Table	Type	D:	SI	Object Name
1	0		0	0	10.340	0,0	0	0	0	0						OLE_FLUSH_CALL
1	0		3	0	9.490	0,0	9.490	0	0	604		TRANSP	AI	4		BALDAT

Graphical Explain Explain With Hint Explain Recompiled Plan **Show More**

SQL Statement

```
SELECT
*
FROM
BALDAT
WHERE
LOG_HANDLE=?
```

Execution Plan (PRECOMPILED)

Operation	Options
PROJECT	BALDAT.MANDANT, BALDAT.RELID, BALDAT.LOG_HANDLE, BALDAT.BLOCK, BALDAT.SRTF2, BALDAT._DATAAGING, BALDAT.CLUSTR, BALDAT.CLUSTD

Operator	Ex. Engine	Table	Operator Details	Operator Properties
PROJECT	HEX		BALDAT.MANDANT, BALDAT.RELID, BALDAT.LOG_HANDLE, BALDAT.BLOCK, BALDAT.SRTF2, BALDAT._DATAAGING, BALDAT.CLUSTR, BALDAT.CLUSTD	PRECOMPILED, PHYSICAL_ENUM_BY: HEX_PROJECT
TABLE SCAN	HEX		BALDAT FILTER CONDITION: BALDAT.LOG_HANDLE = ?1 (DETAIL: ([INDEX LOOK-UP] BALDAT.LOG_HANDLE = ?1))	PHYSICAL_ENUM_BY: HEX_TABLE_SCAN

LOOK-UP!

DETERMINATION AND USAGE OF SAP HANA INDEXES

- After creating explain Plan you can see the explain in the output of the „*HANA_SQL_StatementHash_DataCollector*“ for affected statement
- SCAN is a Full Column Scan
- POST FILTER is, in Oracle terminology, a "table access by index rowid".
- TABLE SCAN can be anything.
- INDEX SCAN operations are different from index lookups in context of TABLE SCAN operations.
 - INDEX SCAN: Produces sorted result set and then filters
 - INDEX LOOK-UP with TABLE SCAN: Filters at first and then sorts / aggregates in the next step
 - SAP Note: [2142945 - FAQ: SAP HANA Hints](#)

Source: SAP

EXPERIENCES WITH SAP HANA INDEXES: CREATE SINGLE-COLUMN INDEX



SELECT

item_t0.PK

FROM TABLE_A item_t0

WHERE (item_t0.p_template = ? **AND**
item_t0.COLUMN_A= ?) **AND** (item_t0.TypePkString=?
) **order by** item_t0.p_date

DESC LIMIT 1 **OFFSET** 0

- Column: COLUMN_A
- NUM_DISTINCT: 4838394
- DATA_TYPE: BIGINT
- COMPRESSION: DEFAULT
- INDEX_TYPE: NONE
- LOAD_UNIT: COLUMN
- SCANNED_RECS_PER_S: 2.488.944.777
- INDEX_LOOKUPS_PER_H: 0

Weeks after creating index:

TABLE_NAME	COLUMN_NAME	SCANNED_RECORDS	SCR_PER_S	INDEX_LOOKUPS	INDEX_LOOKUPS_PER_H
TABLE_A	COLUMN_A	0	0	59446046	111332

EXPERIENCES WITH SAP HANA INDEXES: CREATE SINGLE-COLUMN INDEX

Explain Plan before creating index:

```
TABLE SCAN;  
FILTER CONDITION:  
ITEM_T0.TYPEPKSTRING = ?3  
AND ITEM_T0.COLUMN_A = ?2  
AND ITEM_T0.P_TEMPLATE = ?1  
(DETAIL: ([POST-FILTER] ITEM_T0.  
COLUMN_A = ?2  
AND ITEM_T0.P_CTEMPLATE = ?1  
AND ITEM_T0.TYPEPKSTRING = ?3));
```

Explain Plan after creating index:

```
TABLE SCAN;  
FILTER CONDITION:  
ITEM_T0.TYPEPKSTRING = ?  
AND ITEM_T0.COLUMN_A = ?2 AND  
ITEM_T0.P_TEMPLATE = ?1  
(DETAIL: ([INDEX LOOK-UP]  
ITEM_T0.COLUMN_A = ?2)  
AND ([POST-FILTER]  
ITEM_T0.P_TEMPLATE = ?1  
AND ITEM_T0.TYPEPKSTRING = ?3))
```

EXPERIENCES WITH SAP HANA INDEXES: CREATE SINGLE-COLUMN INDEX

Runtime Comparison: SQL: „HANA_SQL_StatementHash_DataCollector“

SNAPSHOT_TIME	PLAN_ID	ENG	EXECUTIONS	RECORDS	REC_PER_EXEC	EXEC_S	AVG_EXEC_MS	AVG_CPU_MS
CURRENT	226970002	H	550239923	60780469	0.11	70435.51	0.12	0.17
2026/06/07	122978960002	H	14504162	1579477	0.10	2250.74	0.15	0.20
2026/06/06	122978960002	H	11836357	1230592	0.10	1840.34	0.15	0.20
2026/06/05	122978960002	H	22189252	2423171	0.10	3254.90	0.14	0.19
2026/06/04	122978960002	H	22305503	2464116	0.11	3320.23	0.14	0.20
2026/06/03	122978960002	H	9489925	1025571	0.10	1686.72	0.17	0.24
2026/06/03	21920002	H	30481710	3299023	0.10	119780.63	3.92	5.50
2026/06/02	21920002	H	44710128	4993379	0.11	162183.48	3.62	5.30
2026/06/01	21920002	H	13300536	1469812	0.11	35167.27	2.64	4.72
2026/05/31	21920002	H	13685245	1503275	0.10	32325.74	2.36	4.79
2026/05/30	21920002	H	10819922	1126917	0.10	24664.04	2.27	4.66
2026/05/29	21920002	H	21874922	2388941	0.10	50065.53	2.28	4.44
2026/05/28	21920002	H	28507852	3176388	0.11	65201.28	2.28	4.26
2026/05/27	21920002	H	27587968	3055845	0.11	63169.58	2.28	4.32

EXPERIENCES WITH SAP HANA INDEXES: CREATE MULTI-COLUMN INDEX

Before creating multi-column Index

```
...  
DETAIL:  
((([INDEX LOOK-UP] TESTTABLE.PARTNER = ?4) AND  
([SCAN] TESTTABLE.ROLE_PARTNER IN (?5, ?6, ?7)) AND  
([SCAN] TESTTABLE.DELETED <> 'X')) AND  
([POST-FILTER] TESTTABLE.NOT_REL_REASON = '00' AND TESTTABLE.CLIENT = ?1)  
)
```

After creating multi-column Index: "TESTTABLE" (ROLE_PARTNER, PARTNER)

```
....  
(DETAIL:  
TABLE SCAN  
([INDEX LOOK-UP] $ROLE_PARTNERS$PARTNER$ IN (__cs_concat__(?5, ?4), __cs_concat__(?6, ?4), __cs_concat__(?7, ?4))) AND  
([POST-FILTER] TESTTABLE.DELETED <> 'X' AND  
TESTTABLE.NOT_REL_REASON = '00' AND  
TESTTABLE.CLIENT = ?1))
```

EXPERIENCES WITH SAP HANA INDEXES: CREATE MULTI-COLUMN INDEX



Runtime Comparison: SQL: „HANA_SQL_StatementHash_DiffReport“

KEY_FIGURE	VALUE_1	VALUE_2	CHANGE_PCT
Begin time	2026/03/16 08:27:06	RESET (2026/04/16 08:27:06)	
End time	2026/04/14 08:27:06	RESET (2026/04/16 08:27:06)	
Statement hash	13099bf7857bb36fefb46a1c1452923b	13099bf7857bb36fefb46a1c1452923b	
Execution engines	HEX	HEX	
Executions	7662782.00	19443.00	-99.74
Records	492214.00	1451.00	-99.70
Records per execution	0.06	0.07	16.18
Execution time (ms)	1038841670.21	2398.29	-99.99
Execution time (ms) per execution	135.56	0.12	-99.90
Execution time (ms) per record	2110.54	1.65	-99.92
CPU time (ms)	1260893234.96	3131.18	-99.99
CPU time (ms) per execution	164.54	0.16	-99.90
CPU time (ms) per record	2561.67	2.15	-99.91
Called threads	35527194.00	0.00	-100.00
Called threads per execution	4.63	0.00	-100.00
Called threads per record	72.17	0.00	-100.00
Used memory (MB)	942722160.03	818.28	-99.99
Used memory (MB) per execution	123.02	0.04	-99.96
Used memory (MB) per record	1915.26	0.56	-99.97

EXPERIENCES WITH SAP HANA INDEXES: REMOVE MULTI-COLUMN INDEX

```
SELECT
```

```
  "WI_ID" FROM "SWW_WI2OBJ" WHERE  
"CLIENT" = ? AND "CATID" = ? AND  
"INSTID" = ? AND "TYPEID" = ? AND  
"WI_RH_TASK" = ? AND "WI_RELTYPE" =  
N'08' LIMIT 1
```

```
INDEX_NAME COLUMN_NAME INDEX_TYPE  
CONSTRAINT_NAME  
=====
```

SWW_WI2OBJ~0	CLIENT	INVERTED VALUE
	PRIMARY KEY	
	GUID	
SWW_WI2OBJ~H9L	TOP_WI_ID	INVERTED VALUE
SWW_WI2OBJ~HFZ	WI_ID	INVERTED VALUE
SWW_WI2OBJ~HQ7	INSTID	INVERTED VALUE
SWW_WI2OBJ~Y01	INVERTED VALUE	
	CLIENT	
	INSTID	
	CATID	

EXPERIENCES WITH SAP HANA INDEXES: REMOVE MULTI-COLUMN INDEX

System ABC: Good Performance

```
TABLE SCAN
FILTER CONDITION: SWW_WI2OBJ.CLIENT = ?1 AND
SWW_WI2OBJ.WI_RH_TASK = ?5
AND SWW_WI2OBJ.CATID = ?2 AND
SWW_WI2OBJ.INSTID = ?3 AND
SWW_WI2OBJ.TYPEID = ?4 AND
SWW_WI2OBJ.WI_RELTYPE = '08'
(DETAIL: ([INDEX LOOK-UP]
$INSTID$CATID$CLIENT$ = __cs_concat__(?3,
?2, ?1)) AND
([POST-FILTER] SWW_WI2OBJ.WI_RH_TASK = ?5
AND SWW_WI2OBJ.TYPEID = ?4 AND
SWW_WI2OBJ.WI_RELTYPE = '08'))
```

System YXZ: Bad Performance

```
DETAIL:
((([SCAN] SWW_WI2OBJ.WI_RH_TASK = ?5)
AND
([SCAN] SWW_WI2OBJ.TYPEID = ?4) AND
([SCAN] SWW_WI2OBJ.WI_RELTYPE = '08'))
AND
([POST-FILTER]
$CLIENT$INSTID$CATID$GUID$ BETWEEN
__cs_concat__(?1, ?3, ?2) AND
__cs_concat__(?1, ?3, ?2)
)
)
```

EXPERIENCES WITH SAP HANA INDEXES: REMOVE MULTI-COLUMN INDEX

Explain Plan before removing index:

Too many options can lead to mistake: Overchoice:
increased risk of making the wrong decision

DETAIL:

```
(([SCAN] SWW_WI2OBJ.WI_RH_TASK = ?5) AND  
([SCAN] SWW_WI2OBJ.TYPEID = ?4) AND  
([SCAN] SWW_WI2OBJ.WI_RELTYPE = '08')) AND  
([POST-FILTER] $CLIENT$INSTID$CATID$GUID$  
BETWEEN  
__cs_concat__(?1, ?3, ?2) AND __cs_concat__(?1,  
?3, ?2)  
)  
)
```

Other option using Hints:

```
HEX_TABLE_SCAN_INDEX_LOOKUP("SWW_WI2OBJ"."INSTID")  
DEV_HEX_INDEX_LOOKUP("SWW_WI2OBJ"."INSTID")
```

Explain Plan after removing index:

```
TABLE SCAN  
FILTER CONDITION: SWW_WI2OBJ.CLIENT = ?1 AND  
SWW_WI2OBJ.WI_RH_TASK =  
?5 AND SWW_WI2OBJ.CATID = ?2 AND  
SWW_WI2OBJ.INSTID = ?3 AND  
SWW_WI2OBJ.TYPEID = ?4 AND SWW_WI2OBJ.WI_RELTYPE  
= '08' (DETAIL:  
([INDEX LOOK-UP] SWW_WI2OBJ.INSTID = ?3) AND  
([POST-FILTER]  
SWW_WI2OBJ.WI_RH_TASK = ?5 AND SWW_WI2OBJ.TYPEID  
= ?4 AND  
SWW_WI2OBJ.WI_RELTYPE = '08' AND SWW_WI2OBJ.CATID  
= ?2 AND  
SWW_WI2OBJ.CLIENT = ?1))  
)
```

EXPERIENCES WITH SAP HANA INDEXES: REMOVE INDEX

Runtime Comparison: SQL: „HANA_SQL_StatementHash_DiffReport“

KEY_FIGURE	VALUE_1	VALUE_2	CHANGE_PCT
Begin time	2026/03/16 15:16:53	RESET (2026/04/16 15:16:53)	
End time	2026/04/14 15:16:53	RESET (2026/04/16 15:16:53)	
Statement hash	4b9523c2db9464a609790a0633da47b7	4b9523c2db9464a609790a0633da47b7	
Execution engines	HEX	HEX	
Executions	919567.00	1529.00	-99.83
Records	47.00	0.00	-100.00
Records per execution	0.00	0.00	-100.00
Execution time (ms)	375652215.11	364.91	-99.99
Execution time (ms) per execution	408.50	0.23	-99.94
Execution time (ms) per record	7992600.32	0.00	-100.00
CPU time (ms)	1050726825.81	431.32	-99.99
CPU time (ms) per execution	1142.63	0.28	-99.97
CPU time (ms) per record	9999999.99	0.00	-100.00
Called threads	9896997.00	0.00	-100.00
Called threads per execution	10.76	0.00	-100.00
Called threads per record	210574.40	0.00	-100.00
Used memory (MB)	75679602.23	55.52	-99.99
Used memory (MB) per execution	82.29	0.03	-99.95
Used memory (MB) per record	1610204.30	0.00	-100.00

HINTS

- DEV_HEX_INDEX_LOOKUP("<column>")
DEV_HEX_INDEX_LOOKUP("<table>."<column>")
 - With SAP HANA >= 2.0 SPS 08 this hint is replaced by the public HEX TABLE SCAN INDEX LOOKUP hint (SAP Note 3539241).
- CS_FILTER_FIRST("<column>")
- CS_PRIMARY_KEY
- NO_CS_PRIMARY_KEY
- HEX_INDEX_JOIN
NO_HEX_INDEX_JOIN
- HEX_TABLE_SCAN_INDEX_LOOKUP("<table>."<column>")
- HEX_UNIQUE_INDEX_SEARCH
NO_HEX_UNIQUE_INDEX_SEARCH
- HEX_INDEX_SCAN / NO_HEX_INDEX_SCAN
- Row Store:
 - INDEX("<table>" ("<index>"))
 - NO_INDEX("<table>")
- Hints from the SAP note 2700051

TAKEAWAYS

- **Crucial Relevance:** Support for constraints & CPU/query optimization
- **The Golden Rule:** "Less is More" — minimal index count & size
- **Column Store:** Multi-Column vs. single-column (preference) & implicit vs. explicit distinction
- **Over-Indexing Risks:** Higher memory usage & slower write operations & wrong optimizer decisions
- **Cleanup :** Deletion of obsolete indexes
- **Migration:** conversion to Inverted Individual
- **Moderate full column scans are normal:** Do not immediately create new indexes
- **High Scan Analysis:** Investigation of missing indexes vs. unselective queries
- **Runtime Evaluation:** Analysis of statement runtimes instead of instant index creation
- **Mandatory Testing:** Before/after execution plans (including insert, merge, (re)partitioning and optimize compression runtimes)

USEFUL LINKS AND SAP NOTES

- [2160391 - FAQ: SAP HANA Indexes](#)
- [2600076 - FAQ: SAP HANA Inverted Individual Indexes](#)
- [2800008 - FAQ: SAP HANA Fulltext Indexes](#)
- [1794297 - Secondary Indexes for S/4HANA and the business suite on HANA](#)
- [2000002 - FAQ: SAP HANA SQL Optimization](#)
- [2000000 - FAQ: SAP HANA Performance Optimization](#)
- [2114710 - FAQ: SAP HANA Threads and Thread Samples](#)
- [1986747 - How-To: Analyzing internal Columns in SAP HANA Column Store](#)
- [3386070 - How and when to create an index in SAP HANA?](#)
- [3082320 - Columns with many scanned records / Columns with High Number of Scanned Records](#)
- [3481585 - Creating an index for a most scanned table column of a specific SQL statement](#)
- [3624129 - How to create secondary index on HANA DB to help query performance](#)
- [1969700 - SQL Statement Collection for SAP HANA](#)

USEFUL LINKS AND SAP NOTES

- [SAP HANA Performance Guide for Developers | SAP Help Portal](#)
- [openSAP Kurs \(learning.sap.com\): A First Step Towards SAP HANA Query Optimization](#)
- [Deep Diving into SAP HANA SQL Performance Fluctuation](#)
- Blog: [SAP HANA Optimizer - the heart of the performance](#), Jens Gleichmann
- [Best experience: Creation of HANA Indexes part I](#), Jens Gleichmann
- [Best experience: Creation of HANA Indexes part II](#), Jens Gleichmann

THANKS

"No debt is more urgent than that of giving thanks."

(Marcus Tullius Cicero, 106-43 v.Chr., römischer Philosoph)

- HANA Tech Con Community and Jury: Thank you very much for the opportunity to stand on stage alongside all these HANA experts.
- Martin Frauendorfer (SAP) for his HANA SQL-Collection, HANA FAQ Notes and his support
- My special thanks go to Jens Gleichmann for our ongoing and valuable discussions on HANA
- Our customers
- Thank you all for your time

*THANK YOU FOR YOUR
ATTENTION*

